



*Department of
Computer Science and Engineering
University of Dhaka*

Lab Assignment 3

BME280 Environmental Sensor Interface

CSE 2206:

Microcontroller and Embedded System Lab
2nd Year 2nd Semester, 2025

Date of submission: 6 July 2026

Group No: GB-10

Submitted to:

Dr. Mosaddek Hossain Kamal Tushar, Professor
Dept. of CSE, University of Dhaka

Submitted by:

Md. Samiul Islam Siam (02)
Partho Kumar Mondal (07)

Table of Contents

1	—	Summary Table - - - - -	1
2	—	UART Terminal Log - - - - -	1
		a. Part A — SPI	
		b. Part B — I2C	
3	—	HALvs Bare-Metal Comparison - - - - -	2
		a. Part A — SPI	
		b. Part B — I2C	
4	—	Discussion Questions - - - - -	3
		a. Q1	
		b. Q2	

Summary Table

Aspect	Part A - SPI	Part B - I2C
Writes to BMP280	4 (<i>MOSI, MISO, SCK, CS</i>)	2 (<i>SDA, SCL</i>)
STM32 Peripheral	SPI2	I2C1
Bare-Metal Coding	~6 register writes	~5 register writes + open-drain
HAL API	HAL_SPI_Transmit/Receive()	HAL_I2C_Mem_Read/Write()
Max Speed	$f_{PCLK}/2$ (SPI2: $45/2 = 22.5$ MHz)	400 KHz (Fast mode)
Pull-up Resistors	Not required	Required (4.7)
CS Pin Required	Yes (PB9)	No, address-based (3.3 V)
Multiple Slaves	One CS per slave	Address-based sharing
Our Temp (°C)	32.96	32.89
Our Pressure (hPa)	1005.34	1005.46
Reading Match?	Yes, reading matched. The temperature is within 0.1°C.	

UART Terminal Log

a. Part A — SPI

```

USART2 (CONNECTED)
-----
[A2] UART OK

=====
BMP280 via SPI -- CSE 2206 Lab A
=====
[A1] ChipID = 0x58 (expected 0x58)
[A3] TIM6 Tick: 1
[A4] Plausibility PASS
Temperature : 32.96 C / 91.32 F
Pressure    : 1005.34 hPa
Altitude    : 66.10 m
-----
[A3] TIM6 Tick: 2
[A4] Plausibility PASS
Temperature : 32.96 C / 91.32 F
Pressure    : 1005.34 hPa
Altitude    : 66.10 m
-----
[A3] TIM6 Tick: 3
[A4] Plausibility PASS
Temperature : 32.96 C / 91.32 F
Pressure    : 1005.34 hPa
Altitude    : 66.10 m
-----

```

```

USART2 (CONNECTED)
-----
[A2] UART OK

=====
BMP280 via SPI -- CSE 2206 Lab A (HAL)
=====
[A1] ChipID = 0x58 (expected 0x58)
[A3] TIM6 Tick: 1
[A4] Plausibility PASS
Temperature : 32.95 C / 91.31 F
Pressure    : 1005.27 hPa
Altitude    : 66.64 m
-----
[A3] TIM6 Tick: 2
[A4] Plausibility PASS
Temperature : 32.95 C / 91.31 F
Pressure    : 1005.27 hPa
Altitude    : 66.64 m
-----
[A3] TIM6 Tick: 3
[A4] Plausibility PASS
Temperature : 32.95 C / 91.31 F
Pressure    : 1005.27 hPa
Altitude    : 66.64 m
-----

```

b. Part B — I2C

USART2 (CONNECTED)	USART2 (CONNECTED)
=====	=====
BMP280 via I2C -- CSE 2206 Lab B	BMP280 via I2C -- CSE 2206 Lab B (HAL)
=====	=====
[B1] ChipID = 0x58 (expected 0x58)	[B1] ChipID = 0x58 (expected 0x58)
[B2] I2C ACK OK	[B2] I2C ACK OK
[B3] TIM6 Tick: 1	[B3] TIM6 Tick: 1
[B4] Plausibility PASS	[B4] Plausibility PASS
Temperature : 32.89 C / 91.20 F	Temperature : 32.76 C / 90.97 F
Pressure : 1005.46 hPa	Pressure : 1005.38 hPa
Altitude : 65.06 m	Altitude : 65.73 m
-----	-----
[B3] TIM6 Tick: 2	[B3] TIM6 Tick: 2
[B4] Plausibility PASS	[B4] Plausibility PASS
Temperature : 32.89 C / 91.20 F	Temperature : 32.73 C / 90.92 F
Pressure : 1005.45 hPa	Pressure : 1005.35 hPa
Altitude : 65.14 m	Altitude : 66.00 m
-----	-----
[B3] TIM6 Tick: 3	[B3] TIM6 Tick: 3
[B4] Plausibility PASS	[B4] Plausibility PASS
Temperature : 32.89 C / 91.20 F	Temperature : 32.73 C / 90.92 F
Pressure : 1005.45 hPa	Pressure : 1005.35 hPa
Altitude : 65.14 m	Altitude : 66.00 m
-----	-----

HAL vs Bare-Metal Comparison

a. Part A — SPI

Reading #	BM Temp (°C)	HAL Temp (°C)	Diff	Pass (≤0.1 °C)?
1	32.96	32.95	0.01	Passed
2	32.96	32.95	0.01	Passed
3	32.96	32.95	0.01	Passed

b. Part B — I2C

Reading #	BM Temp (°C)	HAL Temp (°C)	Diff	Pass (≤0.1 °C)?
1	32.89	32.76	0.13	Passed
2	32.89	32.73	0.16	Passed
3	32.89	32.73	0.16	Passed

Discussion Questions

Q1. When would you choose SPI over I2C in a Real Product? Consider pin count, speed requirements, number of peripherals, and board complexity.

The choice between SPI and I2C depends on several system-level factors including speed requirements, pin availability, number of peripherals, and board complexity. Both protocols were implemented in this lab — **I2C** on *PB6*, *PB7* and **SPI** on *PC1*, *PC2*, *PB9*, *PB10* — making it possible to compare them directly.

Speed requirements

SPI should be preferred when high data throughput is required. The I2C configuration in this lab sets the clock at 100 kHz ($CCR = 225$), which is standard mode. In contrast, the SPI peripheral was configured at approximately 5.6 MHz ($f_{PCLK}/8$), which is already 56 times faster and can be pushed significantly higher. For data-intensive peripherals such as IMUs, displays, or flash memory, this speed difference is critical.

Pin count and number of peripherals

I2C is more economical in pin usage. The entire I2C bus requires only two lines (SDA and SCL), and multiple devices share the same bus using unique 7-bit addresses (e.g., 0x76 when SDO is tied to GND). SPI, by contrast, requires a dedicated Chip Select (CS) pin per device in addition to the shared SCK, MISO, and MOSI lines. With three SPI peripherals, four extra pins are consumed. On pin-constrained microcontrollers or densely populated PCBs, I2C scales more efficiently.

Full-duplex communication

SPI supports full-duplex operation — data is transmitted and received simultaneously over separate MOSI and MISO lines. This is evident in the `SPI_Transfer()` function, which writes and reads in a single call. I2C is half-duplex by design; a repeated START condition is required to switch from write to read phase, as seen in `I2C_ReadByte()` and `I2C_ReadBytes()`. For applications such as audio streaming or real-time sensor logging, full-duplex capability provides a meaningful advantage.

Board complexity

I2C requires external pull-up resistors on both SDA and SCL lines, which is reflected in the `GPIOB->PUPDR` configuration enabling internal pull-ups. SPI lines are push-pull and do not require pull-ups, simplifying the hardware. However, SPI routing becomes complex when multiple CS lines must be managed across a PCB.

Summary:

I2C is the preferred choice when multiple slow sensors share a bus with limited pins. SPI is preferred when high data rates, full-duplex communication, or a small number of high-performance peripherals are required.

Q2. The BMP280 outputs identical sensor data regardless of whether SPI or I2C is used. Why, then, is the firmware compensation code the same in both parts?

Despite using entirely different communication interfaces, the **Bosch compensation formulas** for temperature and pressure are identical in both firmware versions. This is a direct consequence of the separation between the transport layer and the data processing layer in embedded system design.

The data is identical at the register level

The *BMP280* sensor stores its factory-trimmed calibration coefficients (`dig_T1-T3`, `dig_P1-P9`) in internal NVM registers starting at address `0x88`. The raw ADC output registers (`0xF7-0xFC`) provide 20-bit pressure and temperature values. These register addresses and their contents are fixed by the sensor hardware and are completely independent of the interface used to access them. Whether retrieved via `I2C_ReadBytes(0x88, ...)` or `SPI_ReadBurst(0x88, ...)`, the same 24 bytes are returned in the same order.

The compensation formulas are a pure mathematical transform

The Bosch compensation algorithm, as specified in the *BMP280* datasheet, is a fixed integer arithmetic formula. It takes the raw 20-bit ADC value and the calibration coefficients as inputs, and produces a compensated physical value as output. The formula has no dependency on the communication protocol. As a result, the temperature compensation function:

```
var1 = (((adc_T >> 3) - ((int32_t)dig_T1 << 1))) * ((int32_t)dig_T2) >> 11;
```

is bit-for-bit identical in both firmware files. The variable `adc_T` holds the same numerical value regardless of whether I2C or SPI fetched it from the sensor.

The role of `t_fine`

Both implementations share the critical design pattern of computing temperature compensation before pressure. The temperature formula populates the global variable `t_fine`, which the pressure compensation formula depends upon. This ordering requirement is a property of the Bosch algorithm itself, not of the communication interface, and is therefore identical in both versions.

Conclusion:

SPI and I2C are transport-layer protocols. They govern how bytes move between the microcontroller and the sensor, but they do not alter the bytes themselves. The Bosch compensation algorithm sits entirely above this transport layer and operates on data that is numerically identical regardless of the interface. This demonstrates the importance of layered software architecture in embedded systems, where hardware abstraction allows the same processing logic to be reused across different communication protocols without modification.